

Node.js Practical Assignment -1

Name : Prajapati Krish P.

Roll no : 48

Div : Ict-3

1. Develop nodejs application with following requirements:

- Develop a route `"/gethello"` with GET method. It displays `"Hello NodeJS!!"` as response.
- Make an HTML page and display.
- Call `"/gethello"` route from HTML page using AJAX call. (Any frontend AJAX call API can be used.)

Code: (app.js)

```
const express = require('express');
const path = require('path');
const app = express();
const port = 3000;
app.use(express.static("public"));
app.get("/gethello", (req, res) => {
  res.send("Hello node js");
})
app.listen(port, () => {
  console.log(`Server is running on port ${port}`);
})
```

Index.html

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>node practical</title>
</head>
```

```
<body>

  <h2>Node Js Practical</h2>

  <button onclick="gethello()">get message</button>


  <h3 id="message"></h3>

  <script>

    function gethello() {

      fetch('/gethello')

        .then(response => response.text())

        .then(data => {

          document.getElementById('message').innerText = data;

        });

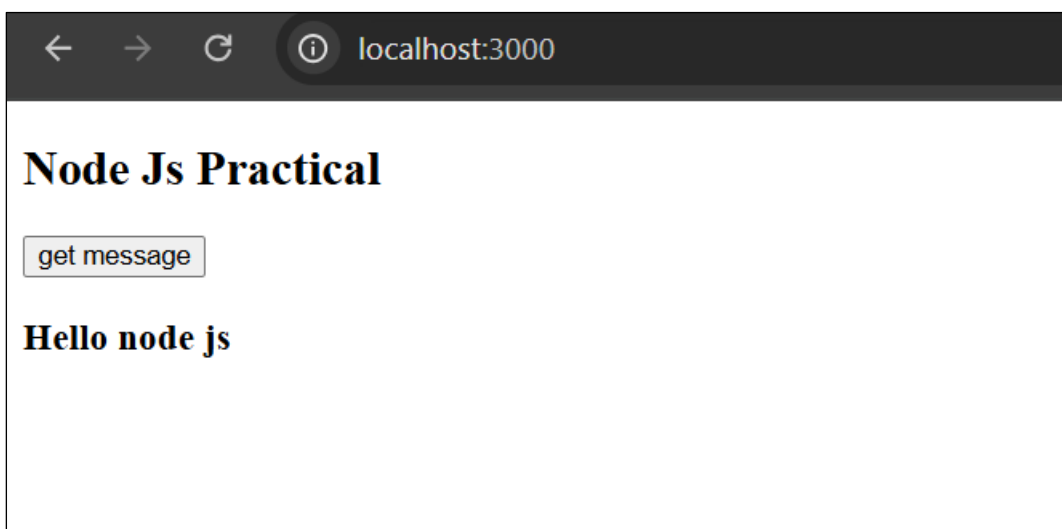
    }

  </script>

</body>

</html>
```

Output :



2. Develop a web server which serves static resources.

Code :- app.js

```
const express = require('express');

const app = express();
app.use(express.static('public'));
app.listen(3000, () =>{
  console.log('Server is running on port 3000');
})
```

Index.html

```
<!DOCTYPE html>

<html>

<head>

  <title>Static Resource Example</title>

  <link rel="stylesheet" href="style.css">

</head>

<body>

<h1>Welcome to Node.js Static Server</h1>



<br><br>
```

```
<button onclick="showMessage()">
```

```
    Click Me
```

```
</button>
```

```
<script src="script.js"></script>
```

```
</body>
```

```
</html>
```

Style.css :

```
body {
```

```
    text-align: center;
```

```
    font-family: Arial;
```

```
    background-color: lightblue;
```

```
}
```

```
h1 {
```

```
    color: darkblue;
```

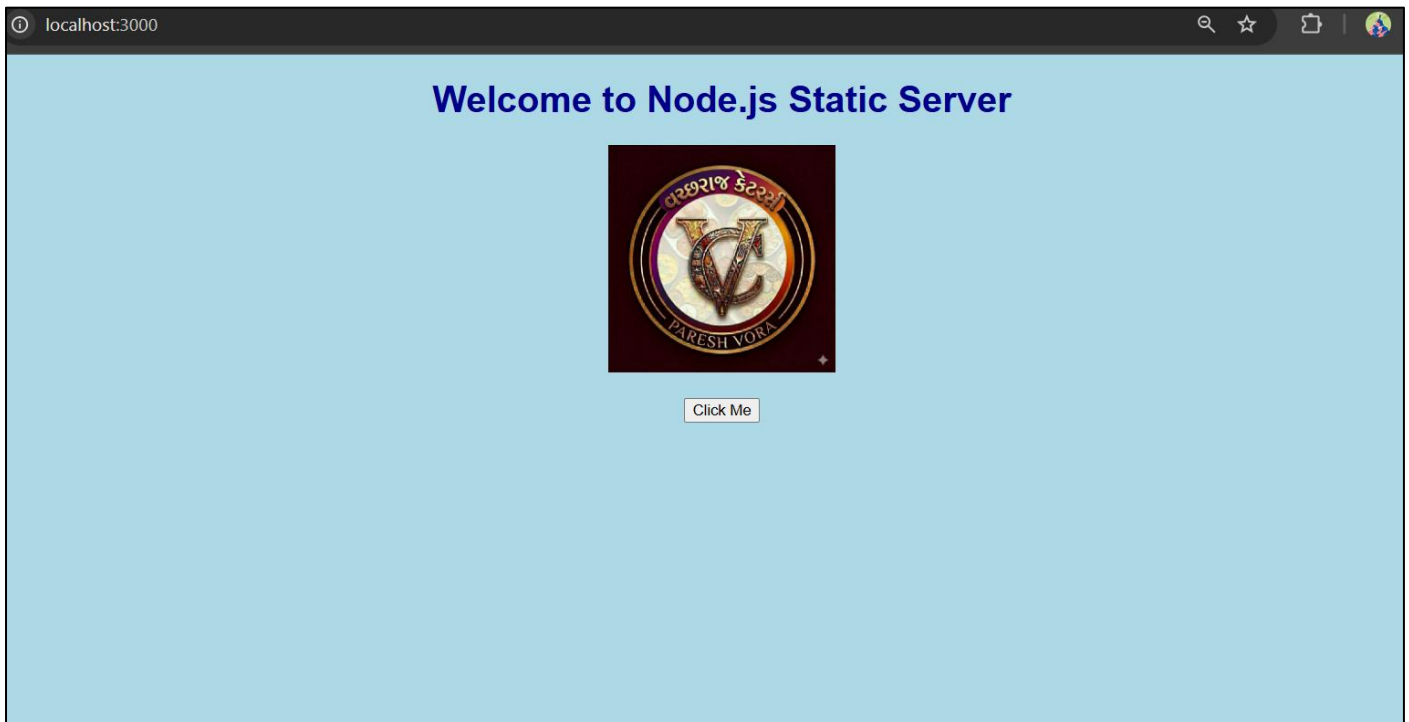
```
}
```

Script.js :

```
function showMessage() {
```

```
    alert("Hello from JavaScript!");
```

```
}
```

OutPut :**3. Develop a module for domain specific chatbot and use it in a command line application.****Code : App.js**

```
const readline = require('readline');  
const getReply = require('./chatbot');  
  
const rl = readline.createInterface({  
  
  input: process.stdin,  
  output: process.stdout  
});  
  
rl.question('ask your question: ', (que) => {  
  const ans = getReply(que);
```

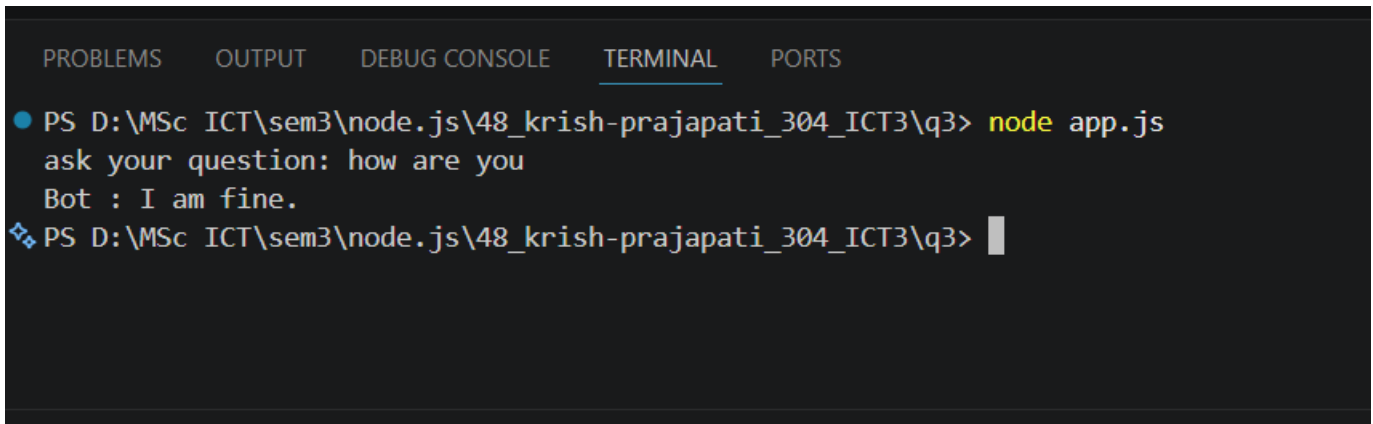
```
    console.log("Bot :", ans);  
  
    rl.close();  
  
});
```

Chatbot.js

```
function getReply(question) {  
    if (question === "hi") {  
        return "Hello!";  
    }  
    else if (question === "how are you") {  
        return "I am fine.";  
    }  
    else if (question === "course") {  
        return "We provide Node.js, Java and Python courses.";  
    }  
    else if (question === "fees") {  
        return "Course fee is ₹5000.";  
    }  
    else if (question === "bye") {  
        return "Thank you! Visit again.";  
    }  
    else {  
        return "I don't understand your question.";  
    }  
}
```

```
module.exports = getReply;
```

output :-



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\MSc ICT\sem3\node.js\48_krish-prajapati_304_ICT3\q3> node app.js
ask your question: how are you
Bot : I am fine.
PS D:\MSc ICT\sem3\node.js\48_krish-prajapati_304_ICT3\q3>
```

4. Write a program to create a compressed zip file for a folder.

Code app.js

```
const fs = require("fs");
```

```
const archiver = require("archiver");
```

```
const output = fs.createWriteStream("data.zip");
```

```
const archive = archiver("zip", {
```

```
  zlib: { level: 9 }
```

```
});
```

```
archive.pipe(output);
```

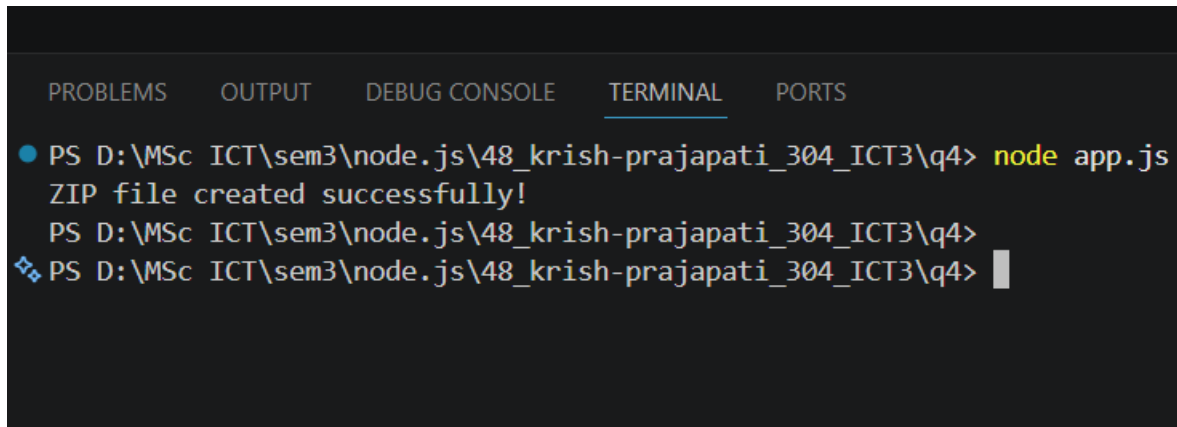
```
archive.directory("data/", false);
```

```
archive.finalize();
```



```
output.on("close", () => {  
    console.log("ZIP file created successfully!");  
});
```

Output :-



The screenshot shows a VS Code terminal window with the 'TERMINAL' tab selected. The terminal displays the following commands and output:

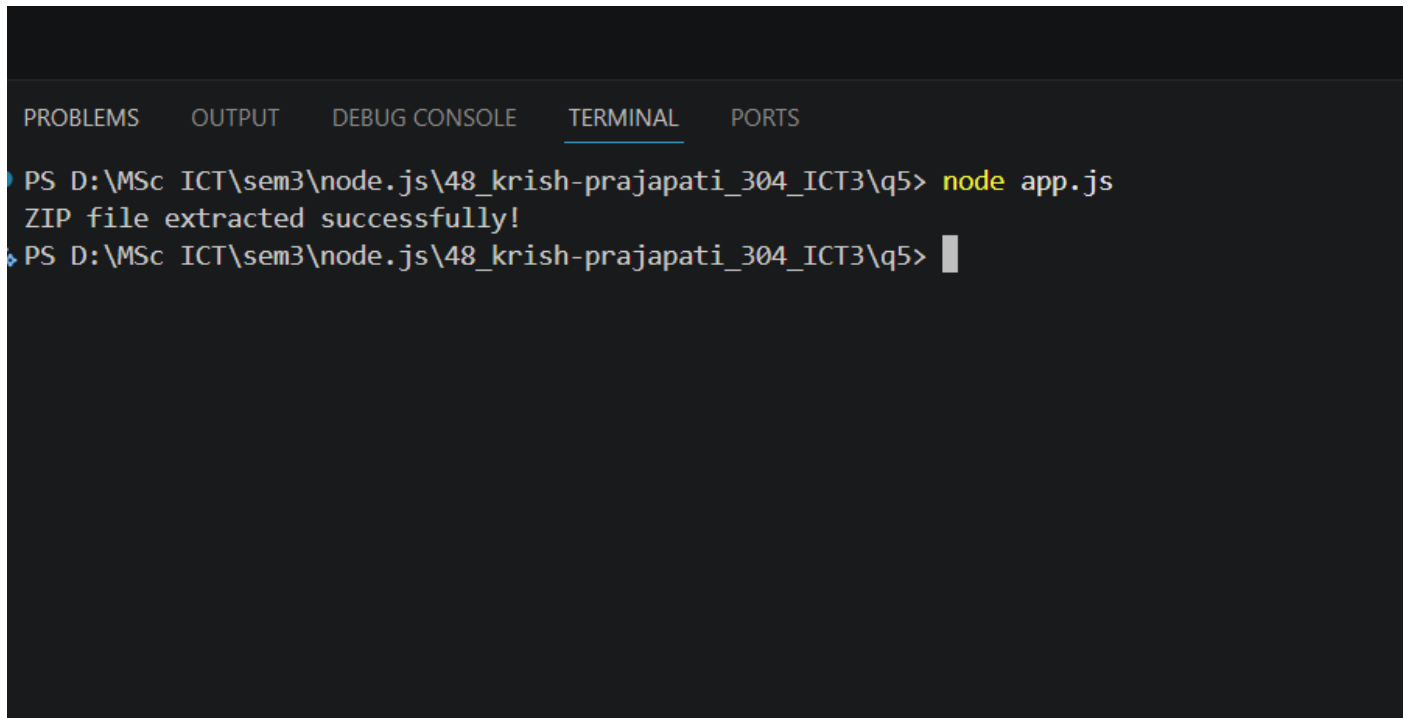
```
PS D:\MSc ICT\sem3\node.js\48_krish-prajapati_304_ICT3\q4> node app.js  
ZIP file created successfully!  
PS D:\MSc ICT\sem3\node.js\48_krish-prajapati_304_ICT3\q4>  
PS D:\MSc ICT\sem3\node.js\48_krish-prajapati_304_ICT3\q4>
```

5. Write a program to extract a zip file.

Code : app.js

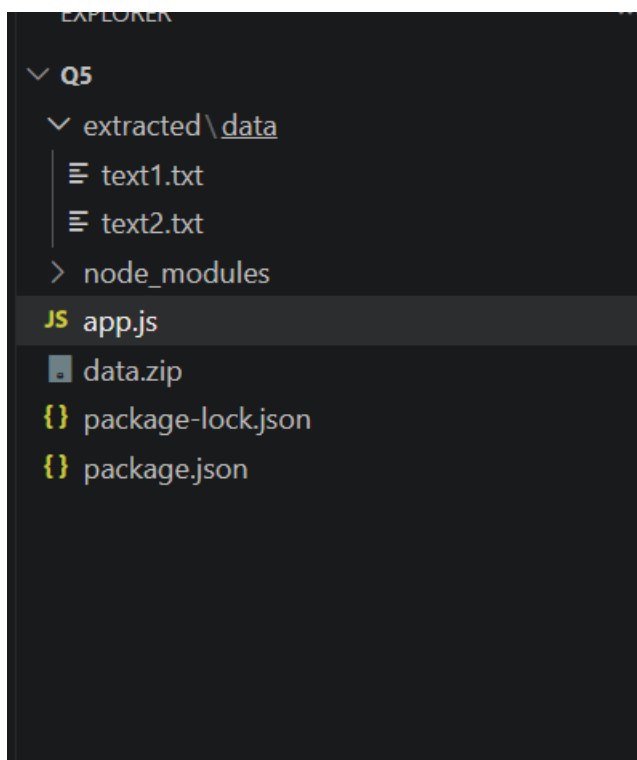
```
const fs = require("fs");  
  
const unzipper = require("unzipper");  
  
// Read the ZIP file  
fs.createReadStream("data.zip")  
  
    .pipe(unzipper.Extract({ path: "extracted" })))  
  
    .on("close", () => {  
        console.log("ZIP file extracted successfully!");  
    });
```

```
});
```

Output :

The screenshot shows a terminal window with a dark background. At the top, there are tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL' (which is active), and 'PORTS'. Below the tabs, the terminal shows the following commands and output:

```
PS D:\MSc ICT\sem3\node.js\48_krish-prajapati_304_ICT3\q5> node app.js
ZIP file extracted successfully!
PS D:\MSc ICT\sem3\node.js\48_krish-prajapati_304_ICT3\q5>
```

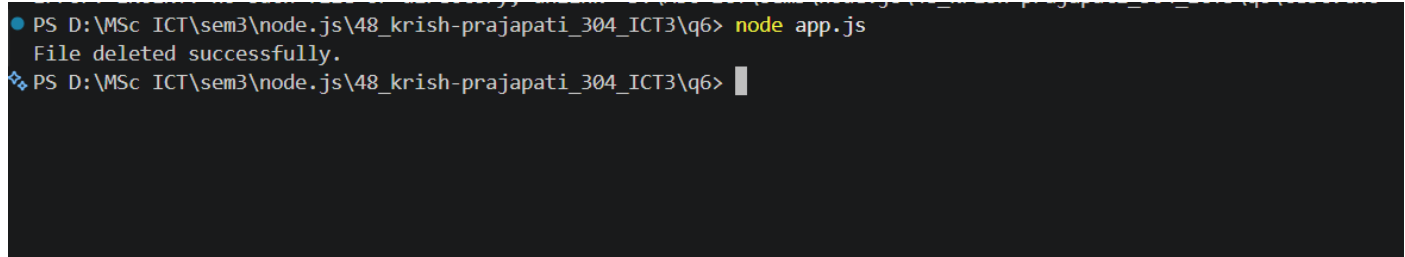


The screenshot shows a file explorer window with a dark background. The left sidebar shows a tree view with the following structure:

- Q5
 - extracted\data
 - text1.txt
 - text2.txt
 - node_modules
 - JS app.js
 - data.zip
 - package-lock.json
 - package.json

6. Write a program to promisify fs.unlink function and call it.**Code : app.js**

```
const fs = require("fs");  
  
const util = require("util");  
  
// Convert callback function into Promise  
const unlinkFile = util.promisify(fs.unlink);  
  
// Call the promisified function  
unlinkFile("test.txt")  
  .then(() => {  
    console.log("File deleted successfully.");  
  })  
  .catch((err) => {  
    console.log("Error:", err.message);  
  });
```

Output :

```
PS D:\MSc ICT\sem3\node.js\48_krish-prajapati_304_ICT3\q6> node app.js  
File deleted successfully.  
PS D:\MSc ICT\sem3\node.js\48_krish-prajapati_304_ICT3\q6>
```

7. Develop a node web server with a route /google which can fetch data of google page using fetch() using async-await model and show as a response.

Code : app.js

```
const express = require("express");

const app = express();

const PORT = 3000;

app.get("/google", async (req, res) => {
  try {
    const response = await fetch("https://www.google.com");

    const data = await response.text();

    res.send(data);

  } catch (error) {
    res.status(500).send("Error: " + error.message);
  }
});

app.get("/search", async (req, res) => {
  const query = req.query.q;

  if (!query) {
    return res.status(400).send("Query parameter 'q' is required.");
  }
});
```

```
}

try {

  const response = await
fetch(`https://www.google.com/search?q=${encodeURIComponent(query)}`);

  const data = await response.text();

  res.send(data);

} catch (error) {

  res.status(500).send("Error: " + error.message);

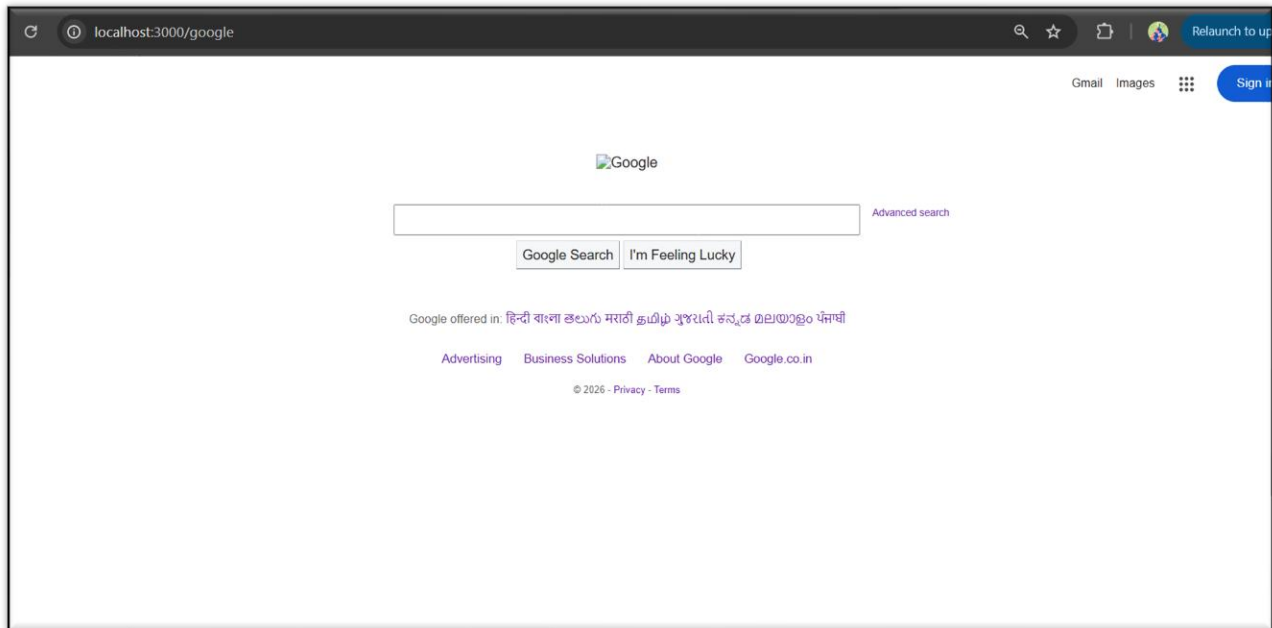
}

});

app.listen(PORT, () => {

  console.log(`Server running at http://localhost:${PORT}`);

});
```

Output :

8. Set a server script, a test script and 3 user defined scripts in package.json file in your nodejs application.

Code :**app.js**

```
console.log("Server script is running...");
```

hello.js :

```
console.log("Hello User!");
```

info.js :

```
console.log("Info Script Running...");
```

test.js :

```
console.log("Test Script Running...");
```

welcome.js :

```
console.log("Welcome to Node.js");
```

package.json :

```
{  
  "name": "q8",  
  "version": "1.0.0",  
  "description": "",  
  "main": "app.js",  
  "scripts": {  
  
    "start": "node app.js",  
    "test" : "node test.js",  
  
    "info": "node info.js",  
    "welcome" : "node welcome.js",  
    "hello" : "node hello.js"  
  },  
  "keywords": [],  
  "author": "",  
  "license": "ISC",  
  "dependencies": {  
    "express": "^5.2.1"  
  }  
}
```

Output :

```
Node.js v22.19.0
PS D:\MSc ICT\sem3\node.js\48_krish-prajapati_304 ICT3\q8> npm start

> q8@1.0.0 start
> node app.js

Server script is running...
PS D:\MSc ICT\sem3\node.js\48_krish-prajapati_304 ICT3\q8> npm test

> q8@1.0.0 test
> node test.js

Test Script Running...
PS D:\MSc ICT\sem3\node.js\48_krish-prajapati_304 ICT3\q8> npm run welcome

> q8@1.0.0 welcome
> node welcome.js

Welcome to Node.js
PS D:\MSc ICT\sem3\node.js\48_krish-prajapati_304 ICT3\q8> npm run hello

> q8@1.0.0 hello
> node hello.js

Hello User!
PS D:\MSc ICT\sem3\node.js\48_krish-prajapati_304 ICT3\q8> npm run info

> q8@1.0.0 info
> node info.js

Info Script Running...
```

9. A program which calls useful functions in fs module.**Code : app.js**

```
const fs = require('fs');
```

```
console.log('Starting the application...');
```

```
fs.writeFileSync("demo.txt", "Hello Node.js");
```

```
const data = fs.readFileSync("demo.txt", "utf8");
```

```
console.log("File Content:", data);
```



```
fs.appendFileSync("demo.txt", "\nWelcome to FS Module");
```

```
console.log("\nAfter Append:");
```

```
console.log(fs.readFileSync("demo.txt", "utf8"));
```

```
fs.renameSync("demo.txt", "sample.txt");
```

```
console.log("\nFile Renamed Successfully");
```

```
if (fs.existsSync("sample.txt")) {
```

```
    console.log("File Exists");
```

```
}
```

```
const stats = fs.statSync("sample.txt");
```

```
console.log("File Size:", stats.size, "bytes");
```

```
fs.unlinkSync("sample.txt");
```

```
console.log("File Deleted Successfully");
```

output :

```
PS D:\MSc ICT\sem3\node.js\48_krish-prajapati_304_ICT3\q9> node app.js
Starting the application...
File Content: Hello Node.js

After Append:
Hello Node.js
Welcome to FS Module

File Renamed Successfully
File Exists
File Size: 34 bytes
File Deleted Successfully
PS D:\MSc ICT\sem3\node.js\48_krish-prajapati_304_ICT3\q9> 
```

10. A program which uses global objects in nodejs. Also read command line arguments and print it.

Code : app.js

```
// Global Object: console
console.log("=== Global Objects Demo ===");

console.log("Current Directory:");
console.log(__dirname);

console.log("\nCurrent File:");
console.log(__filename);

console.log("\nNode Version:");
console.log(process.version);

console.log("\nPlatform:");
console.log(process.platform);
```

```
// Read Command Line Arguments
```

```
console.log("\nCommand Line Arguments:");
```

```
const args = process.argv.slice(2);
```

```
if (args.length === 0) {
```

```
    console.log("No arguments passed.");
```

```
} else {
```

```
    args.forEach((arg, index) => {
```

```
        console.log(`Argument ${index + 1}: ${arg}`);
```

```
    });
```

```
}
```

```
setTimeout(() => {
```

```
    console.log("\nProgram Finished.");
```

```
}, 2000);
```

Output :

```
PS D:\MSc ICT\sem3\node.js\48_krish-prajapati_304_ICT3\q10> node app.js krish vora Ict3
=== Global Objects Demo ===
Current Directory:
D:\MSc ICT\sem3\node.js\48_krish-prajapati_304_ICT3\q10

Current File:
D:\MSc ICT\sem3\node.js\48_krish-prajapati_304_ICT3\q10\app.js

Node Version:
v22.19.0

Platform:
win32

Command Line Arguments:
Argument 1: krish
Argument 2: vora
Argument 3: Ict3

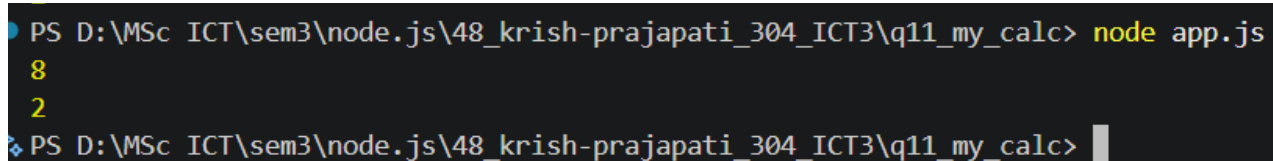
Program Finished.
```

11. Develop a useful package and publish it on npmjs.com. Write a program to use it.**Code : app.js**

```
const calc = require('./index');  
  
console.log(calc.add(5, 3));  
  
console.log(calc.subtract(5, 3));
```

Use It (index.js) :-

```
function add(a, b) {  
    return a + b;  
}  
  
function subtract(a, b) {  
    return a - b;  
}  
  
module.exports = {  
    add,  
    subtract  
};
```

Output :

```
PS D:\MSc ICT\sem3\node.js\48_krish-prajapati_304_ICT3\q11_my_calc> node app.js  
8  
2  
PS D:\MSc ICT\sem3\node.js\48_krish-prajapati_304_ICT3\q11_my_calc>
```

12. Write a node program having at least 3 dependent tasks and call using chained promises.

Code : app.js

```
function task1() {  
  return new Promise((resolve) => {  
    setTimeout(() => {  
      console.log("Task 1 read the data :");  
      resolve("Data");  
    }, 1000);  
  });  
}  
  
function task2(data) {  
  return new Promise((resolve) => {  
    setTimeout(() => {  
      console.log("Task 2 : Processing the data : " + data);  
      resolve("Processed Data");  
    }, 1000);  
  });  
}  
  
function task3(data) {  
  return new Promise((resolve) => {  
    setTimeout(() => {  
      console.log("Task 3: Finalizing the Data : " + data);  
      resolve("Final Data");  
    }, 1000);  
  });  
}
```

```
    }, 1000);  
  
  })  
  
}
```

```
task1()  
  .then(task2)  
  .then(task3)  
  .then((result) => {  
    console.log("final Result : " + result);  
  })  
  .catch((error) => {  
    console.error("Error: " + error);  
  });
```

Output :

```
PS D:\MSc ICT\sem3\node.js\48_krish-prajapati_304_ICT3\q12> node app.js  
Task 1 read the data :  
Task 2 : Processing the data : Data  
Task 3: Finalizing the Data :Processed Data  
final Result : Final Data  
PS D:\MSc ICT\sem3\node.js\48_krish-prajapati_304_ICT3\q12> █
```